

TP3 – Un framework multithreadé

minimaliste

1. Introduction - préparation

1. Programme en polling

On veut coder un programme permettant de faire clignoter un chenillard de LED selon un motif, qui détecte la pression de boutons pour modifier le motif et sa vitesse, et qui affiche sur un écran le nom du motif et sa vitesse.

On commence par une ébauche en mode polling : une boucle infinie vient « scruter » en permanence l'état des boutons et des LEDs.

```
void loop() {  
  
    // Lecture des boutons  
    if (ReadBTNMotif()) {  
        // Changer Le motif actuel  
    }  
    if (ReadBTNSpeedUp()) {  
        // Augmenter La vitesse  
    }  
    if (ReadBTNSpeedDown()) {  
        // Diminuer La vitesse  
    }  
  
    // Calcul du motif à afficher sur les LEDs selon la vitesse et le mode  
    displayLED(motif);  
  
    // Affichage des infos sur l'écran OLED  
    displayOnScreen(infoString);  
  
    // Attente liée à la vitesse  
    HAL_Delay(vitesse);  
}
```

Ce programme pose un problème : chaque boucle dure le temps de la pause due à la fonction **HAL_Delay**. Il est alors impossible d'effectuer d'autres actions en parallèle, notamment détecter l'appui d'un bouton par l'utilisateur.

2. Mise en place du projet de base.

Le projet a été initialisé et l'appel de la fonction `printf` ajouté dans le code main est fonctionnel.

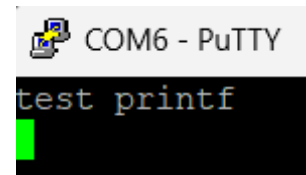


Fig.1 – test de la fonction `printf` réussi

3. Le scheduler, usage du timer.

1. Calcul des paramètres du timer

En allant dans l'onglet « Clock configuration » du fichier `.ioc`, on note que les deux « timer clocks » APB1 et APB2 sont réglés à $F_{APB} = 80\text{MHz}$.

En regardant ensuite la configuration de TIM4 (présent dans la section « Timers » de l'onglet « Pinout & Configuration »), on peut voir les valeurs écrites dans les registres Prescaler et AutoReload Register du timer. On note ici $PSC = 79$ et $ARR = 9999$.

On peut alors calculer la période T_{sch} du scheduler :

$$T_{sch} = \frac{(PSC+1) \times (ARR+1)}{F_{APB}} \quad \underline{\text{AN}} : T_{sch} = \frac{(79+1) \times (9999+1)}{80 \cdot 10^6} = 10\text{ms}$$

Sachant que la liaison série est réglée au débit $D = 115200 \text{ bits/seconde}$, on peut en déduire le nombre de bits N_b envoyé sur la période du scheduler :

$$N_b = D \times T_{sch} \quad \underline{\text{AN}} : N_b = 115200 \times 10 \cdot 10^{-3} = 1152 \text{ bits}$$

Si on considère qu'il y a un bit de start et un bit de stop avant chaque octet, il faut alors 10 *bits* pour transmettre un caractère. Le plus grand nombre de caractères N_c qui peut être envoyé sur une période est donc :

$$N_c = \frac{N_b}{10} \quad \underline{\text{AN}}: N_c = \frac{1152}{10} = 115,2 \cong 115 \text{ caractères}$$

Ainsi, s'il y a trois tâches à réaliser, la période d'exécution de chacune sera égale à la période du scheduler qui est ici de 10ms et elles seront donc finies en 30ms.

A présent, on crée un fichier `task.h` pour définir une fonction pour chacune des trois tâches qui nous seront utiles pour le projet :

```
#ifndef INC_TASK_H_
#define INC_TASK_H_

void taskLED();
void taskButton();
void taskScreen();

#endif /* INC_TASK_H_ */
```

On vient ensuite démarrer les interruptions avant la boucle infinie et après les différentes fonctions de démarrage dans la fonction `main` :

```
/* USER CODE BEGIN 2 */
HAL_TIM_Base_Start_IT(&htim4);
/* USER CODE END 2 */
```

Enfin, on modifie la fonction **HAL_TIM_PeriodElapsedCallback** présente dans le fichier `main.c` pour appeler les différentes tâches dans l'ordre :

```
/* USER CODE BEGIN Callback 0 */
static int taskNumber=0;
if(htim->Instance == TIM4)
{
    taskNumber=(taskNumber+1)%3;
    switch (taskNumber){
        case 0 :
            taskLED();
            break;
        case 1 :
            taskButton();
            break;
        case 2 :
            taskScreen();
            break;
    }
}
/* USER CODE END Callback 0 */
```

4. La tâche de gestion des LEDs

1. Le driver logiciel des LEDs

On inclue les fichiers fournis pour l'utilisation des LEDs.

On vient compléter la fonction **LED_Update** pour que chaque LED du chenillard soit allumé ou non selon la valeur du champ `isOn` de la case correspondante dans le tableau `led_bar` :

```
void LED_Update(){
    for(int i=0; i<LED_BAR_SIZE; i++){
        HAL_GPIO_WritePin(
            led_bar[i].port,
            led_bar[i].pinNumber,
            led_bar[i].isOn
        );
    }
}
```

2. La tâche taskLED()

On copie dans le fichier `task.c` les déclarations indiquées dans l'énoncé.

L'énoncé propose également un calcul pour un entier `numero_motif` à réaliser au début de la fonction `taskLED`. Le calcul vient incrémenter l'entier à chaque fois que l'horloge globale (obtenue avec `HAL_GetTick`) augmente de `globalDelayInMs` millisecondes (initialisé à 200 dans les déclarations précédentes), et retombe à 0 dès qu'il dépasse la taille du motif actuel.

On se sert donc de ce calcul et des drivers des LEDs pour compléter la fonction `taskLED` :

```
#include "led.h"

void taskLED(){
    int numero_motif=
        (HAL_GetTick()/globalDelayInMs)
        %(tableau_motif[index_tableau_motif]->size);
    LED_Set_Value_With_Int(
        tableau_motif[index_tableau_motif]->motif[numero_motif]
    );
    LED_Update();
}
```

5. La tâche de gestion des boutons

Après avoir rajouté la ligne `#include "main.h"` dans `button.h`, on peut compléter la fonction **BUTTON_Update** en suivant la consigne de l'énoncé :

```
#include <stm32l4xx_hal.h>

void BUTTON_Update() {
    for(int i=0; i<5; i++){
        int nouvelle_valeur =
            (HAL_GPIO_ReadPin(joystick[i].port, joystick[i].pinNumber)
             ==GPIO_PIN_RESET)
            ? 1
            : 0;
        joystick[i].hasBeenPressed = (1-joystick[i].isOn)*nouvelle_valeur;
        joystick[i].isOn = nouvelle_valeur;
    }
}
```

1. La tâche taskButton()

On prend soin, comme suggéré par l'énoncé, de trouver un moyen de rafraîchir l'écran uniquement lorsque cela est nécessaire après l'appui d'un bouton, pour ne pas avoir à rafraîchir l'écran à chaque passage dans la boucle et donc risquer de prendre plus de temps que la période du scheduler. Pour cela on déclare une variable `needDisplayRefresh` pouvant valoir 0 s'il n'y a rien à modifier, 1 s'il faut modifier l'affichage du nom du motif courant ou 2 s'il s'agit de la vitesse.

```
#include "button.h"
#define NB_TOTAL_MOTIF 3

int needDisplayRefresh=0;
#define REFRESH_MOTIF 1
#define REFRESH_SPEED 2

void taskButton(){
    BUTTON_Update();
    if(BUTTON_Get_Pressed(CENTER)==1){
        index_tableau_motif++;
        index_tableau_motif%=NB_TOTAL_MOTIF;
        needDisplayRefresh=REFRESH_MOTIF;
    }
    if(BUTTON_Get_Pressed(DOWN)==1){
        globalDelayInMs+=10;
        needDisplayRefresh=REFRESH_SPEED;
    }
    if(BUTTON_Get_Pressed(UP)==1 && globalDelayInMs>10){
        globalDelayInMs-=10;
        needDisplayRefresh=REFRESH_SPEED;
    }
}
```

6. La tâche de gestion de l'écran

Pour l'affichage de l'écran, on commence par une première initialisation juste avant la fonction **HAL_TIM_PeriodElapsedCallback** dans le main afin que l'écran puisse être entièrement affiché une première fois avant qu'il y ait des interruptions par le timer.

```
#include "ssd1315.h"
#include "fonts.h"
#include <stdio.h>

void screen_setup(){

    ssd1315_Init();
    ssd1315_Clear(SSD1315_COLOR_BLACK);

    ssd1315_Draw_String(0, 0, "Welcome ENSEA", &Font_7x10);

    char speed_buffer[16]={0};
    snprintf(speed_buffer, 16, "SPEED= %d", globalDelayInMs);
    ssd1315_Draw_String(0, 10, speed_buffer, &Font_7x10);

    char motif_buffer[16]={0};
    snprintf(motif_buffer, 16, "%s",
            tableau_motif[index_tableau_motif]->name);
    ssd1315_Draw_String(0, 20, motif_buffer, &Font_7x10);

    ssd1315_Refresh();
}
```


On peut ensuite écrire la fonction **taskScreen** qui va pouvoir dynamiquement rafraichir les zones de l'écran qui ont besoin d'être modifiées

```
void taskScreen(){
    if(needDisplayRefresh==REFRESH_SPEED){
        char speed_buffer[16]={0};
        snprintf(speed_buffer, 16, "SPEED= %d", globalDelayInMs);
        ssd1315_Draw_String(0, 10, speed_buffer, &Font_7x10);
        ssd1315_Refresh();
    }
    if(needDisplayRefresh==REFRESH_MOTIF){
        char motif_buffer[16]={0};
        snprintf(motif_buffer, 16, "%s",
                tableau_motif[index_tableau_motif]->name);
        ssd1315_Draw_String(0, 20, motif_buffer, &Font_7x10);
        ssd1315_Refresh();
    }
    needDisplayRefresh=0;
}
```

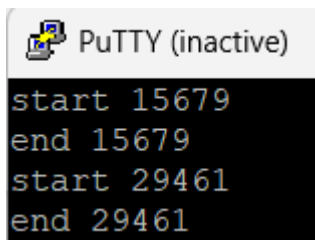


Fig.2 – Mesure du temps de mise à jour des informations sur l'écran OLED

Un affichage du retour de la fonction **HAL_GetTick** au début à la fin d'une mise à jour de l'écran permet de constater que notre système est efficace : la mise à jour est pratiquement instantanée, pas de risque de dépasser la période du scheduler.

Finalement le cahier des charges est pleinement respecté : les LEDs clignotent bien selon le motif et la vitesse qui ont été choisis, l'écran affiche bien le nom du motif courant ainsi que la vitesse, et l'appui sur le bouton central permet de passer au prochain motif tandis que les boutons du haut et du bas permettent respectivement d'augmenter et diminuer la vitesse.

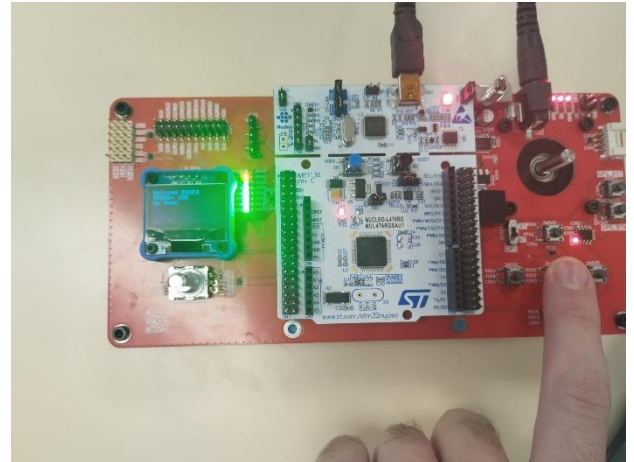
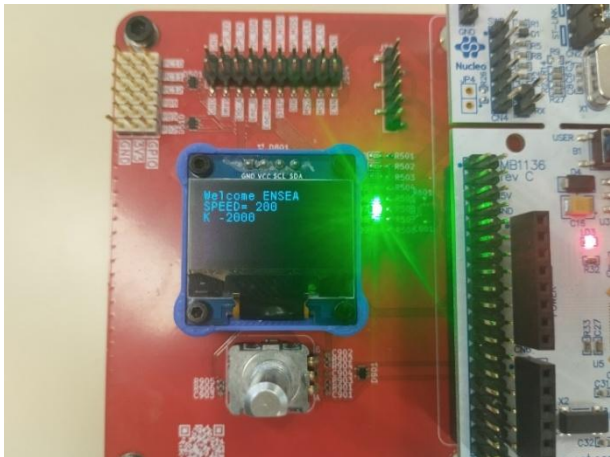


Fig. 3 et 4 – Photo du projet final opérationnel et conforme au cahier des charges

Nous aurions voulu aller plus loin en permettant de choisir le motif précédent et suivant avec les boutons gauche et droite, plutôt que de devoir sélectionner « à la volée » avec le bouton central, et faire de celui-ci un affichage des commandes des boutons sur l'écran OLED, bien que le temps ne nous ait pas permis d'aller au bout de nos idées.